Notebook

February 8, 2026

1 RSA Broadcast attack

So, imagine that you use public exponent $e = 3$ (it was quite popular a long time ago), but you only choose messages that wrap the modulus N, so there is no option of taking cubic root. Is it actually safe? Well, there is a very simple scenario that shows that it's not. Imagine that you have a client that uses RSA to send messages to your server. But they also send messages to several other servers. Say, 2 more. The same message gets encrypted three times: with your public key, with another person's public key and with one more key.

$$C_1 = M^3 \ mod \ N_1$$

$$C_2 = M^3 \ mod \ N_2$$

$$C_3 = M^3 \ mod \ N_3$$

It is next to impossible to break each C_i on its own, but with three, Marvin can solve the puzzle.

1.1 Chinese remainder theorem

If one knows the remainders of Euclidean division of an integer n by several integers, then one can determine uniquely the remainder of the division of n by the product of these integers, under the condition that the divisors are pairwise coprime.

So how can we use this theorem? First, we need to check, that all the divisors (N_1, N_2, N_3) are pairwise coprime. Well, if they are not coprime, then we can find the greatest common divisor of non-coprime ones and factor them, which would allow us to decrypt the message. So let's assume that they are. This means, that we can find such X, that:

$$X < N_1 N_2 N_3$$

$$X = C_1 \ mod \ N_1$$

$$X = C_2 \ mod \ N_2$$

$$X = C_3 \ mod \ N_3$$

and also such X is unique. Let's look at $C = M^3$. Since $M < N_1$ and $M < N_2$ and $M < N_3$, then $C = M^3 < N_1 N_2 N_3$. Also:

$$C = C_1 \ mod \ N_1$$

$$C = C_2 \ mod \ N_2$$
$$C = C_3 \ mod \ N_3$$

So by using Chinese Remainder Theorem we can find C, and all that's left is to take a cubic root.

1.2 How to find C

Let $N_i, i = 1, k$ be divisors and $c_i, i = 1, k$ their respective remainders. $N = N_1 N_2 ... N_k$, and $M_i = N/N_i$ Then $C = (\sum_{i=1}^{k} C_i M_i (M_i^{-1} \ mod \ N_i)) \ mod \ N$

Use the equation to find C and get the flag from the three ciphertexts that you'll receive from the server. Good luck!

[ ]:
```
import socket
import re
from Crypto.Util.number import inverse,long_to_bytes,bytes_to_long
class VulnServerClient:
    def __init__(self,show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1339))
        if show:
            print (self.recv_until().decode())
    def recv_until(self,symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_keys(self,show=True):
        """Receive public keys from the server"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e1=int(re.search('(?<=e1: )\d+',response).group(0))
        N1=int(re.search('(?<=N1: )\d+',response).group(0))
        e2=int(re.search('(?<=e2: )\d+',response).group(0))
        N2=int(re.search('(?<=N2: )\d+',response).group(0))
        e3=int(re.search('(?<=e3: )\d+',response).group(0))
        N3=int(re.search('(?<=N3: )\d+',response).group(0))

        return [(e1,N1),(e2,N2),(e3,N3)]
```

```
    def get_ciphertexts(self,show=True):
        """Receive ciphertexts from the server"""
        self.s.sendall('ciphertext\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        c1=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext1:␣
↪)[0-9a-f]+',response).group(0)))
        c2=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext2:␣
↪)[0-9a-f]+',response).group(0)))
        c3=bytes_to_long(bytes.fromhex(re.search('(?<=ciphertext3:␣
↪)[0-9a-f]+',response).group(0)))
        return (c1,c2,c3)

    def __del__(self):
        self.s.close()
```

[ ]:
```
vs=VulnServerClient()
pk_list=vs.get_public_keys()
(c1,c2,c3)=vs.get_ciphertexts()
```

[ ]:

[ ]: